



Propuesta de Arquitectura

Plataforma global de venta de boletos para eventos en vivo

System Design Challenge — Cloud Engineer Senior

Autor	Rolando Palermo Rodríguez Cruz
Fecha	06 de septiembre de 2026
Versión	1.0
Clasificación	Propuesta técnica

Control de cambios

Autor	Fecha	Descripción
Rolando Palermo Rodríguez Cruz	06/09/2026	Versión inicial de la propuesta: arquitectura global, decisiones técnicas e infraestructura como código / CI/CD.

Índice

Control de cambios	1
1. Diagrama de arquitectura	2
1.1. Contexto y alcance	2
1.2. Arquitectura lógica	3
1.3. Flujos críticos	3
1.3.1. Flash crowd y control de admisión	3
1.3.2. Concurrencia de inventario	5
1.3.3. Pago, boleto y conciliación	7
2. Decisiones importantes	9
2.1. Inventario como fuente de verdad	9
2.2. Modelo de estado del asiento	9
2.3. Picos de tráfico	9
2.4. Consistencia de pago	9
2.5. Transactional outbox e idempotencia	10
2.6. Multi-región y resiliencia	10
2.7. Observabilidad	10
2.8. Tradeoffs principales	11
3. Infraestructura como código & CI/CD	11
3.1. Estructura de Terraform	11
3.2. Versionado y cambios	12
3.3. Secretos	12
3.4. Pipeline CI/CD	12
3.5. Operabilidad	13
Conclusión	13

1. Diagrama de arquitectura

La arquitectura está pensada para operar en cuatro países, absorber picos de 10 a 20 veces el tráfico habitual y proteger dos invariantes: no vender dos veces el mismo asiento y no capturar un pago sin boleto.

1.1. Contexto y alcance

LiveTix opera en México, Estados Unidos, España y Japón. La propuesta usa tres regiones de AWS:

Región AWS	Mercados
us-east-1	México y Estados Unidos
eu-west-1	España
ap-northeast-1	Japón

- La capa global *Route 53* + *CloudFront* + *WAF* es única.
- Los servicios de aplicación y los datos transaccionales se despliegan por región.
- Cada evento tiene una *home region*.
- Las lecturas no críticas (catálogo de eventos, información del evento, imágenes/banners, recomendaciones, etc.) pueden resolverse cerca del usuario.
- La reserva y la venta de asientos siempre se escriben en la región propietaria del evento.

1.2. Arquitectura lógica

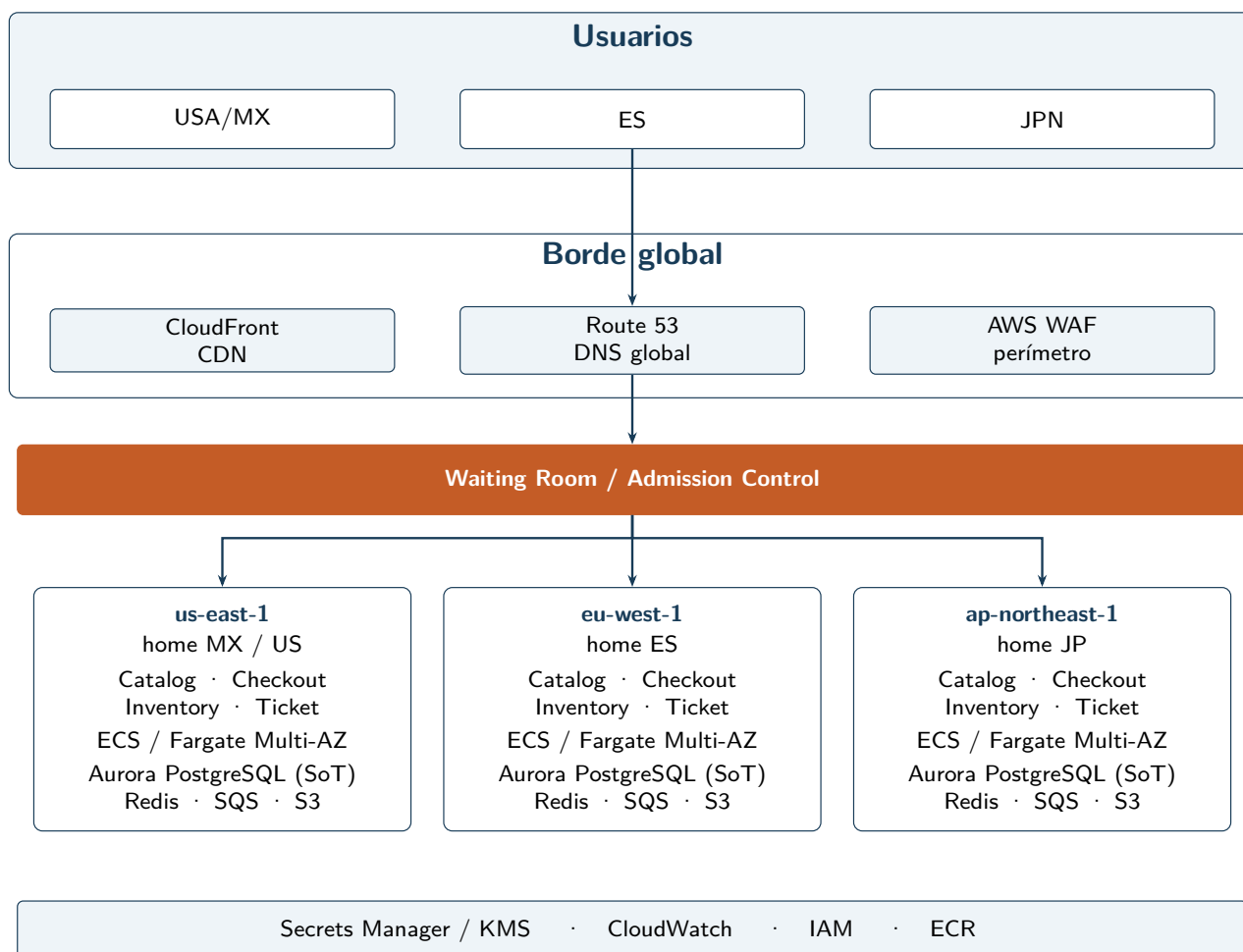


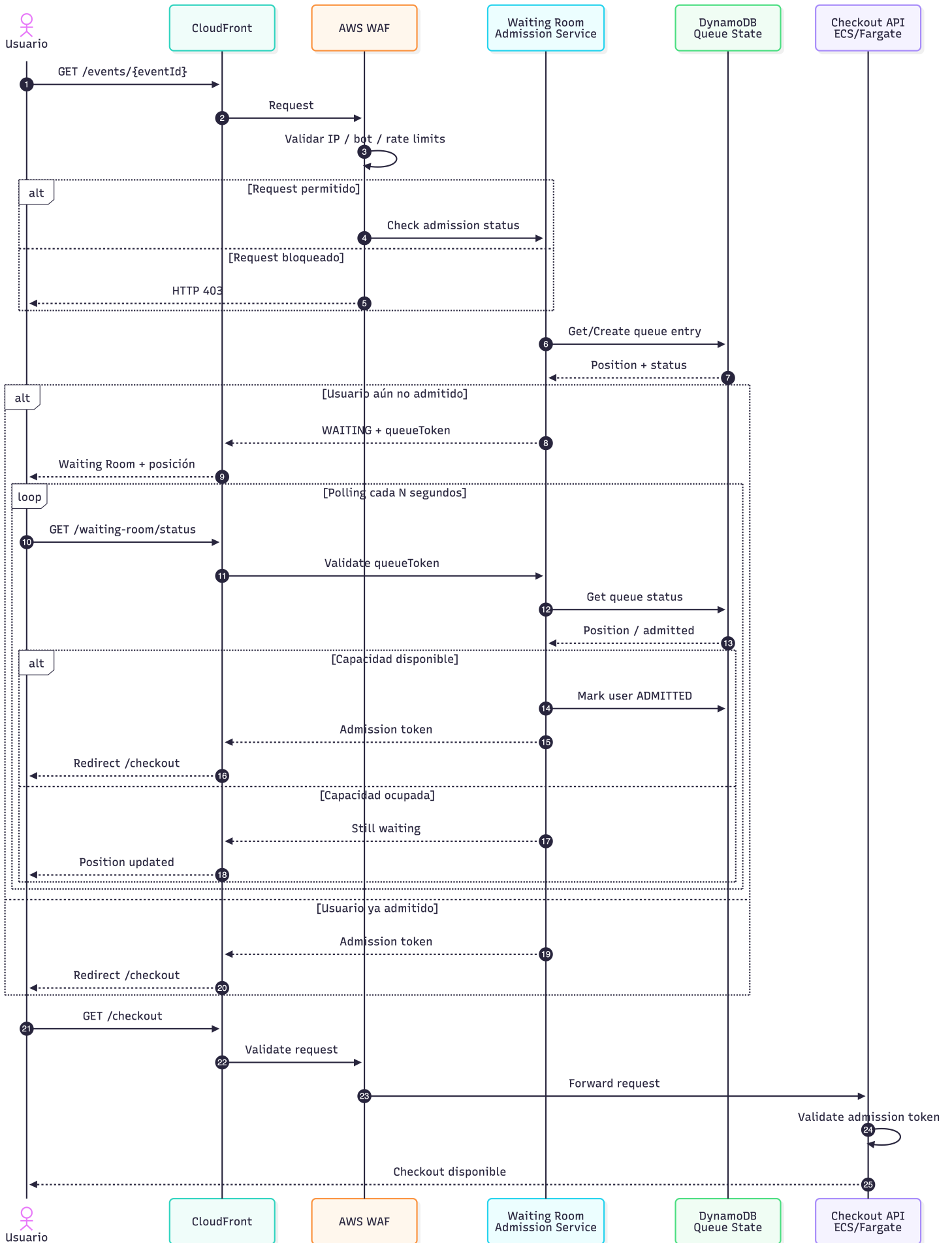
Figura 1: Arquitectura global de LiveTix

1.3. Flujos críticos

1.3.1. Flash crowd y control de admisión

Cuando se liberan boletos, el sistema no espera a que ECS reaccione, en su lugar:

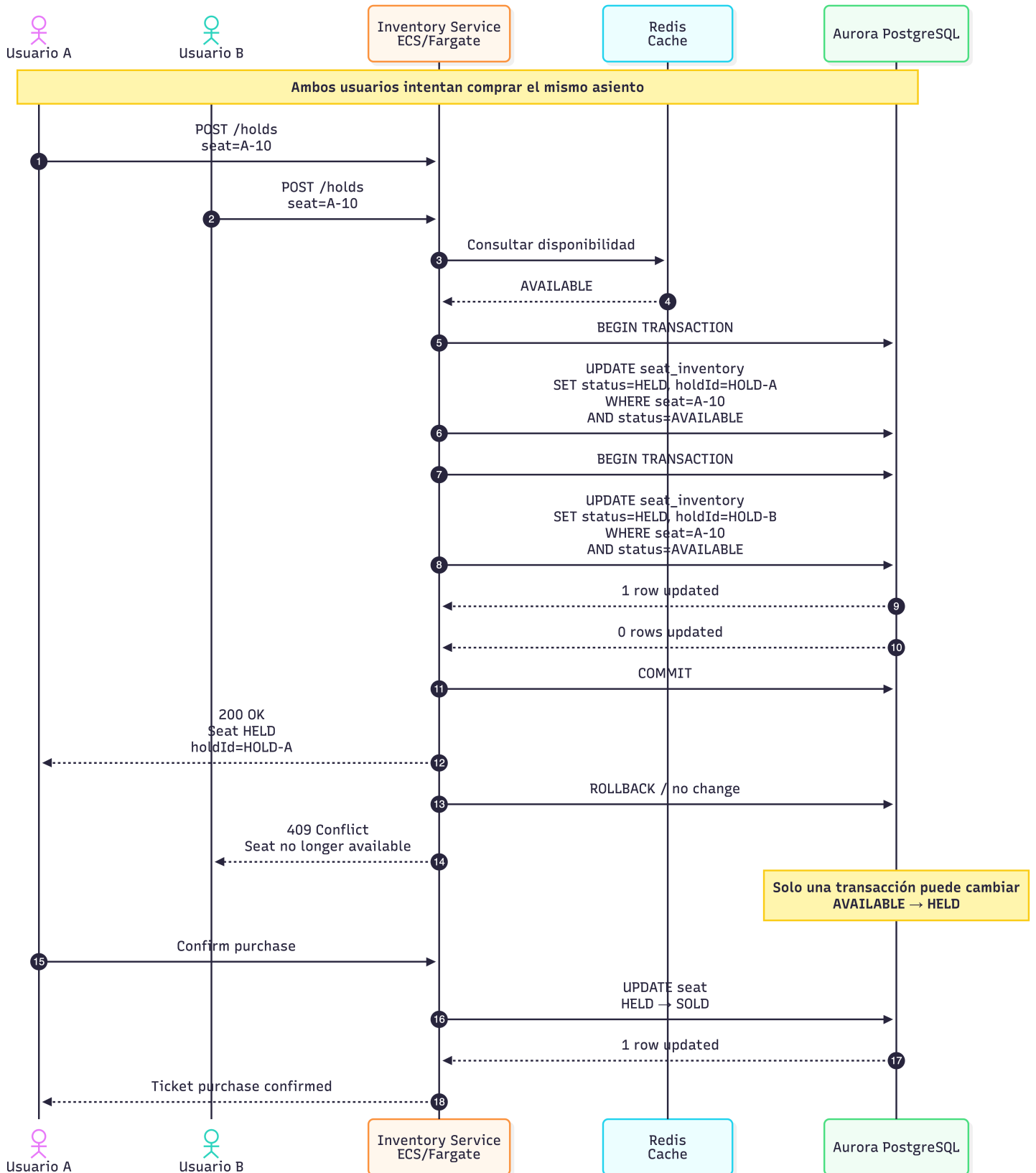
- CloudFront y WAF absorben y filtran.
- La sala de espera dosifica el ingreso al checkout.
- La capacidad de Fargate se preescala antes del evento.



1.3.2. Concurrencia de inventario

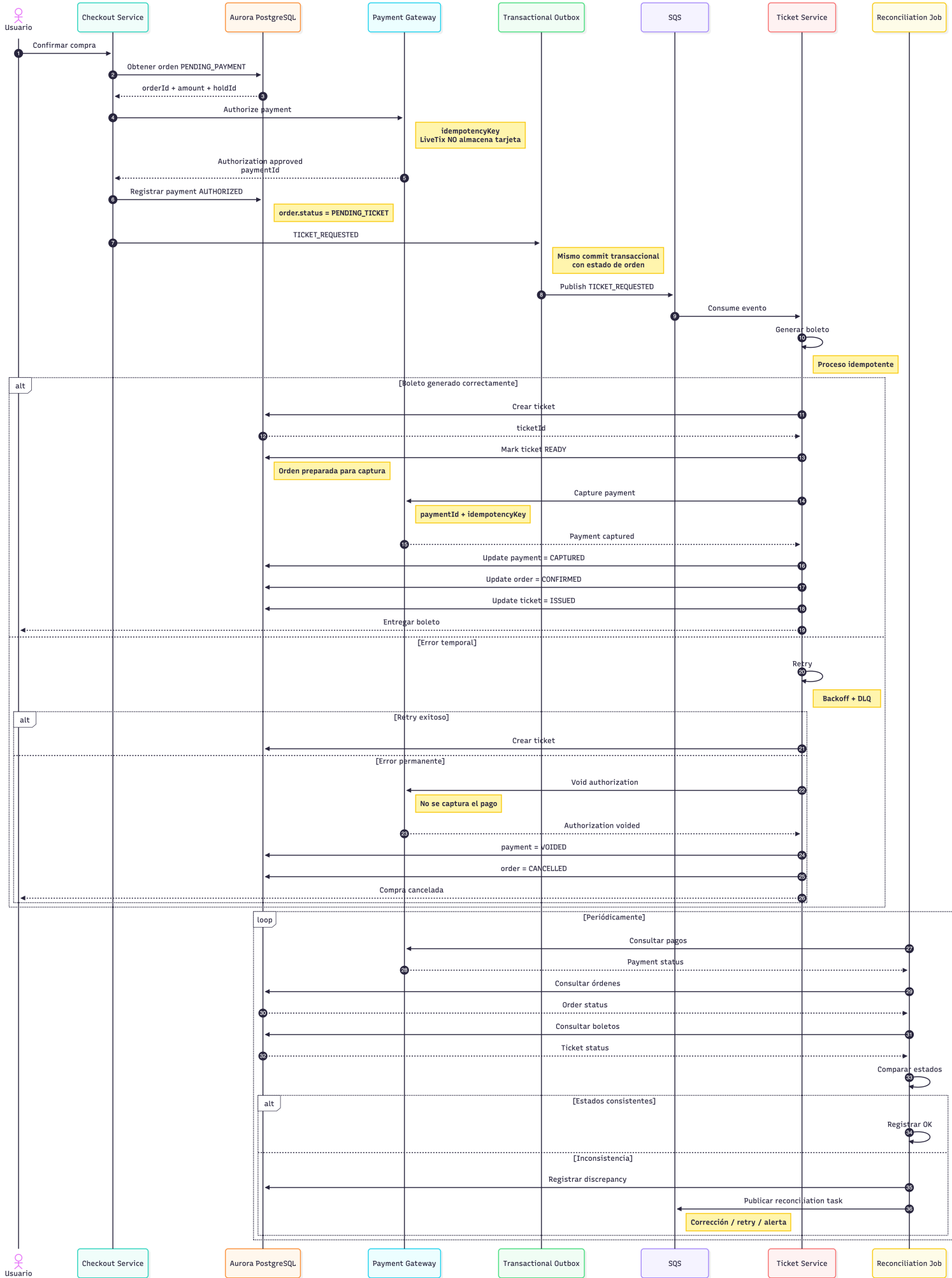
La reserva es una actualización condicional en Aurora. El asiento pasa de **AVAILABLE** a **HELD** y después a **SOLD**, o regresa a **AVAILABLE** si el hold expira.

- El `hold_id` es el token de propiedad temporal.
- El worker de expiración es idempotente: no puede liberar un hold nuevo que ya substituyó al anterior.
- Redis puede mostrar un mapa de asientos más rápido, pero el último asiento lo confirma PostgreSQL.



1.3.3. Pago, boleto y conciliación

- La pasarela autoriza el pago; LiveTix no almacena tarjetas.
- La captura ocurre solo cuando el boleto ya se generó.
- Si la emisión falla de forma permanente, se hace *void* de la autorización y la orden queda cancelada.
- Un reconciliador compara periódicamente pagos, órdenes y boletos.



2. Decisiones importantes

Las decisiones privilegian consistencia, absorción de picos y una operación que un equipo de cinco ingenieros pueda sostener.

2.1. Inventario como fuente de verdad

Aurora PostgreSQL es la autoridad del asiento. El inventario numerado exige exclusión mutua. Una actualización condicional (`status = AVAILABLE`) más una restricción de unicidad evitan la doble venta sin un protocolo distribuido adicional.

Redis no es autoridad. Sirve para cache del mapa de asientos, sesiones y rate limiting. Si Redis se vacía o se desincroniza, el checkout sigue consultando Aurora. Un cache caliente nunca confirma una venta.

2.2. Picos de tráfico

El autoescalado reactivo llega tarde cuando la demanda crece en segundos. La defensa es en capas:

- preescalado de Fargate antes de eventos anunciados;
- CloudFront para lo estático y cacheable;
- WAF en el perímetro;
- waiting room para dosificar checkout;
- SQS para trabajo asíncrono que no debe competir con la reserva.

El objetivo no es que todos lleguen al mismo tiempo al asiento, sino que quienes pasan el cupo encuentren un sistema estable.

2.3. Consistencia de pago

La pasarela es dueña del instrumento; LiveTix es dueño de la orden, el asiento y el boleto. La integración usa claves de idempotencia y webhooks firmados.

Secuencia:

1. Crear hold del asiento.
2. Crear orden en `PENDING_PAYMENT`.
3. Autorizar el pago.
4. Confirmar asiento y orden en una transacción ACID.
5. Registrar `TICKET_REQUESTED` en el transactional outbox.
6. Generar el boleto de forma idempotente.
7. Capturar el pago.
8. Marcar el boleto como emitido.

Autorizar y capturar van separados a propósito: no se cobra en firme si todavía no hay boleto.

2.4. Transactional outbox e idempotencia

El evento de emisión se persiste en la misma transacción que confirma asiento y orden. Un publisher aparte lo coloca en SQS. Los consumidores asumen entrega *at-least-once*: la generación del boleto y la captura deben ser idempotentes.

2.5. Multi-región y resiliencia

Cada región corre en varias Availability Zones. La escritura crítica no cruza de región en el camino feliz. Ante una caída regional completa se recupera en una región secundaria; RPO y RTO se confirman con pruebas de failover, no solo con el diseño.

2.6. Observabilidad

Alarmas prioritarias:

- tasa de éxito de checkout;
- conflictos de reserva de inventario;
- fallos de autorización y de captura;
- latencia y errores de emisión de boletos;
- profundidad y antigüedad de SQS;
- órdenes sin boleto;
- pagos sin boleto correspondiente;
- errores por región y por Availability Zone.

La métrica de negocio que no puede quedar en verde con mentiras es `payment_ticket_inconsistency = 0`.

2.7. Tradeoffs principales

Decisión	Beneficio	Costo
<i>Home region</i> por evento	Consistencia fuerte del inventario	Una compra puede cruzar región
Aurora como fuente de verdad	Evita double-selling	Más costo que un cache puro
Waiting room SQS + outbox	Protege el checkout en picos Absorbe carga y sobrevive a fallos parciales	El usuario espera Más piezas que operar
Fargate	Menos servidores que administrar	Menos control fino que EC2

Decisión	Beneficio	Costo
Multi-AZ + DR regional	Resiliencia real	Costo y ejercicios de failover
Sin K8s ni service mesh	Operable por un equipo pequeño	Menos portabilidad multi-cloud

3. Infraestructura como código & CI/CD

Terraform y el pipeline deben permitir repetir ambientes, aislar regiones y desplegar sin convertir la plataforma en un segundo producto para un equipo de cinco personas.

3.1. Estructura de Terraform

```

infrastructure/
|
+-- modules/
|   +-- networking/
|   +-- ecs/
|   +-- aurora/
|   +-- redis/
|   +-- sqs/
|   +-- cloudfront/
|   +-- observability/
|
+-- environments/
|   +-- dev/
|   +-- staging/
|   +-- production/
|       +-- us-east-1/
|       +-- eu-west-1/
|       +-- ap-northeast-1/
|
+-- global/
    +-- route53/
    +-- cloudfront/
    +-- iam/

```

Los módulos expresan el patrón reutilizable (servicio ECS, cluster Aurora, cola, observabilidad). Los environments solo pasan capacidad, red y flags. Producción se parte por región para limitar el blast radius. Lo global —Route 53, CloudFront, IAM— vive fuera de los stacks regionales.

3.2. Versionado y cambios

Versiones de módulos y providers van fijadas. Todo cambio entra por pull request, con **terraform plan** en CI y apply solo después de revisión. No hay consola como fuente de verdad.

3.3. Secretos

Nada de secretos en git ni en `tfvars` versionados. Secrets Manager y KMS, con IAM de mínimo privilegio y roles de tarea en lugar de access keys estáticas.

3.4. Pipeline CI/CD

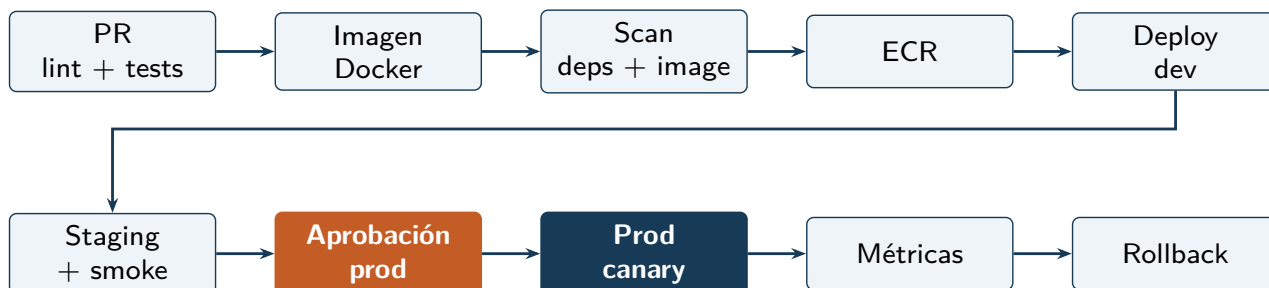


Figura 2: Pipeline: verificación automática hasta staging; producción con aprobación, canary y rollback por alarmas.

Etapas:

1. Commit / pull request.
2. Compilación y pruebas unitarias.
3. Análisis estático y quality gates.
4. Construcción de imagen Docker.
5. Escaneo de dependencias e imagen.
6. Publicación en ECR.
7. Despliegue automático a `dev`.
8. Pruebas de integración y smoke tests.
9. Despliegue a `staging`.
10. Aprobación para producción.
11. Despliegue canary o blue/green.
12. Verificación de métricas y rollback si hay alarma.

Infraestructura sigue un pipeline paralelo: `fmt`, `validate`, `plan` en el PR y `apply` controlado por ambiente. Las migraciones de base de datos deben ser compatibles con el código anterior para no romper un canary a medias.

3.5. Operabilidad

No se introduce Kubernetes, service mesh ni un segundo stack de observabilidad sin una necesidad concreta. El núcleo es ECS/Fargate, Aurora, SQS, CloudFront, Route 53, Secrets Manager y CloudWatch.

Sí se acepta complejidad donde el negocio la exige: outbox, reconciliación y *home region*. El resto se mantiene deliberadamente simple.

Conclusión

La propuesta no maximiza sofisticación. Busca tres garantías: el checkout no colapsa en un flash crowd, el último asiento se vende una sola vez, y una captura de pago tiene boleto.

La *home region* da una fuente de verdad clara sin un inventario activo-activo global. Outbox, consumidores idempotentes y reconciliación cubren los huecos entre LiveTix y la pasarela. Terraform, ambientes aislados y un pipeline con canary permiten operar esto con un equipo pequeño.